

0.2. A, pris dans ALPHABET à l'indice 0

L'alphabet de la console est un tableau : sa première case est l'indice 0.

1. Ce qu'il faut comprendre

`ALPHABET` est le tableau des 26 lettres de la console. Il existe déjà : on ne le crée pas, on s'en sert.

En C++, un tableau commence à l'indice 0. `ALPHABET[0]` est donc la **première** lettre, A ; `ALPHABET[25]` est la dernière, Z.

`poser(0, 0, ALPHABET[0])` pose cette lettre dans la case (0, 0).

Pourquoi `poser()` et pas `texte()` ? `ALPHABET[0]` n'est pas un texte : c'est un **nombre**, le numéro de la tuile du A (1). `texte()` n'accepte que des lettres entre guillemets ; pour mettre une tuile à partir de son numéro, c'est `poser()`. Écrire `texte(0, 0, ALPHABET[0])` est une erreur : le compilateur la refuse.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction `poser\(\)` — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **175 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage — la mise en route de la console
main()         13 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
CopierTuiles   31 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         16 o ← CopierTuiles — 1 lettre sur 43
```

Lettres gravées : A

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 1      6 o sur place, dans main()
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

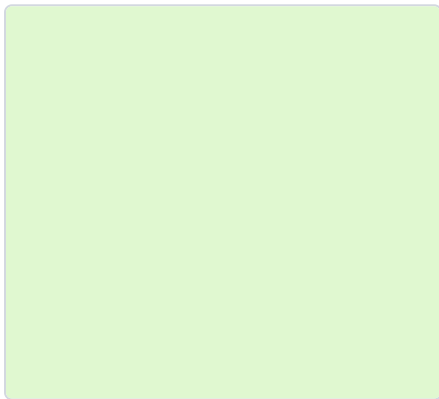
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(0, 0, ALPHABET[0]); // l'indice 0 : la première lettre, A
    // (ne pas faire cette erreur : texte(0, 0, ALPHABET[0]) est refusé,
    // car ALPHABET[0] est un numéro de tuile, pas un texte)

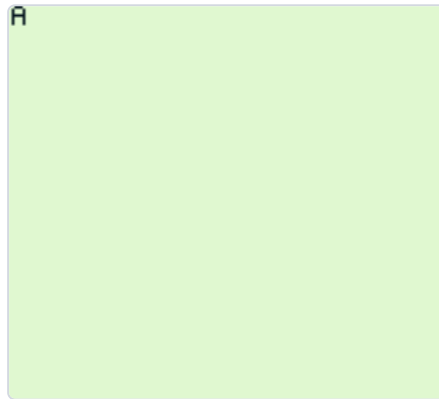
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A, tout en haut à gauche de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ ALPHABET[0] pose un A en (0, 0)

✓ la case d'à côté est vide

Ensuite : **0.2.1. A, pris dans ALPHABET à l'indice 0 — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++